

LogPulse AI - Incident Report

Generated : 2026-05-29 16:42:29 UTC

Trace ID : 4cbc7ee5-a0f6-480f-b4b9-5e3cea13015e

1 - Incident Summary

Incident ID : 4cbc7ee5
Category : Unknown
Severity : SeverityLevel.INFO
Source : Fallback
Cache : MISS

2 - Raw Log

16:42:05 ERROR worker-api memory usage critical: 94% of 256Mi limit, GC thrashing

3 - Classification

Category : Unknown
Severity : SeverityLevel.INFO
Summary : Fallback classification triggered due to inference failure.
Source : Fallback

4 - Diagnosis

Root Cause : The worker-api service is configured with an insufficient memory limit (256Mi) for its current workload, causing it to experience critical memory pressure and GC thrashing.
Confidence : 95%
Reasoning : The raw log explicitly states 'worker-api memory usage critical: 94% of 256Mi limit, GC thrashing', directly indicating the service is hitting its configured memory limit and suffering performance degradation due to insufficient resources.
Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Container OOMKilled Despite Low Reported Memory Usage' (RAG confidence=0.5484). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. SSH to the node running the problematic pod.
- 2. Inspect raw cgroup memory usage for the container: ``cat /sys/fs/cgroup/memory/kubepods/.../memory.usage_in_bytes`` (adjust path). Compare with ``memory.limit_in_bytes``.
- 3. Check ``dmesg -T | grep 'Out of memory'`` for the specific process killed.
- 4. Temporarily increase the container's memory ``limit`` to see if the OOM kill ceases.
- 5. Investigate if the application is creating large amounts of shared memory or ``tmpfs`` mounts, which are part of cgroup memory but not always in ``RSS``.
- 6. Consider upgrading the node kernel, ``kubelet``, or container runtime if known bugs related to cgroup accounting exist for the current versions.

Commands:

```
$ SSH to the node running the problematic pod.  
$ dmesg -T | grep 'Out of memory'  
$ kubectl patch deployment <deployment-name> -p  
'{"spec":{"template":{"spec":{"containers":[{"name":"<container-name>","resources":{"limits":{"memory":"4Gi"}}}}}}}'
```

6 - Risk Assessment

Risk Level : **RISKLEVEL.SAFE**

Safe Commands (3):

```
[OK] SSH to the node running the problematic pod.  
[OK] dmesg -T | grep 'Out of memory'  
[OK] kubectl patch deployment <deployment-name> -p  
'{"spec":{"template":{"spec":{"containers":[{"name":"<container-name>","resources":{"limits":{"memory":"4Gi"}}}}}}}'
```

Flagged Commands (0):

(none)

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED

Executed At : 2026-05-29T13:42:27.956483+00:00

Sandbox Results:

Command : SSH to the node running the problematic pod.

Exit : 0

Time : 0ms

Output:

```
[auto-approve] simulated: SSH to the node running the problematic pod.
```

Command : dmesg -T | grep 'Out of memory'

Exit : 0

Time : 0ms

Output:

[auto-approve] simulated: dmesg -T | grep 'Out of memory'

Command : `kubectl patch deployment <deployment-name> -p`
`'{"spec":{"template":{"spec":{"containers":[{"name":"<container-name>","resources":{"limits":{"memory":"4Gi"}}}]}}}}'`

Exit : 0

Time : 0ms

Output:
[auto-approve] simulated: `kubectl patch deployment <deployment-name> -p`
`'{"spec":{"template":{"spec":{"containers":[{"name":"<container-name>","resources":{"limits":{"memory":"4Gi"}}}]}}}}'`

8 - Historical Context

Times Seen : 1

First occurrence -- no prior incidents.

9 - Incident Timeline

[13:42:27] WORKFLOW_COMPLETED -> Completed in 22605ms